Example Project For Testing Development Strategies

CS162 – Menus, Structures, and IO

Introduction

This program will test several key C++ concepts, mainly **structs**, **input**/**output**, as well as structuring your IO via **menus**.

Prerequisites

Strong understanding of:

- Conditional statements.
- Looping mechanisms.
- Functions.
- Logical operands.
- Arrays.

Basic introduction to:

- Input/output.
- Structures.

Objectives

- Learn how to *bundle*, *access*, *update*, and *pass* data around in a program using **structures**.
- Apply at least one of the popular methods to **input**/**output** text data *to*/*from* the user via standard input/output.
- Format the program to present the directives through a **menu interface**. This will allow you to create testable and repeatable programs in the future.

Background

You have a friend who loves music, but doesn't want to store their library online. They want a custom program to save their music library with. They it to save to/from a text file, but also tell them what's happening as they use the program, with a menu interface.

They don't need anything super fancy, just a basic interface keeping track of basic metadata for individual tracks. No artist linking, no albums/EPs/LPs, just track info.

Requirements

General requirements:

- Saving/loading to/from a text file.
- A menu interface letting user test every function.
- Store *at most* **1024** tracks in an array.

Functions:

- **Display** all tracks.
- **Display** a single track.
- **Update** a track.
- **Delete** a track.
- **Add** a new track.
- **Save** track-list to file.
- **Load** track-list from file.

Track metadata:

- Stored in a **structure**.
- Has the following fields in the struct:
  - Name.
  - Artist.
  - Length.
  - Genre.
  - Album.

Compilation

When it’s time to compile the program, ensure it compiles with the following command/flags:

```
g++ -std=c++23 -g -Wall *.cpp -o <your_username>_test_assignment.out
```

Ensure that it compiles with no errors and no warnings.